5.b Equality

Structural and Referential Equality in Kotlin

Contents

1 **Structural Equality**
Using == and != operators, translation to equals() calls

2 **Referential Equality**
Using === and !== operators to check object identity

3 **Custom equals()**
Overriding equals() and hashCode() for custom equality logic

4 **Floating-point Comparison**
IEEE 754 vs generic equality for NaN and -0.0

Structural Equality

Structural equality verifies if two objects have the same content or structure.
Checked by the == operator and its negated counterpart !=.

How == is translated:

Expression

a == b is translated to:
a?.**equals**(b) ?: (b === null)

Meaning

If a is not null, calls equals(Any?)
If a is null, checks b is also null

Null Comparison

a == null is automatically
translated to a === null

Note

Has nothing to do with Comparable
Only equals(Any?) affects == behavior

Structural Equality – Example

```
fun main() {
    var a = "hello"
    var b = "hello"
    var c = null
    var d = null
    var e = d

    println(a == b)   // true
    println(a == c)   // false
    println(c == e)   // true
}
```

Custom equals() Override

```
class Point(val x: Int, val y: Int) {

    override fun equals(other: Any?): Boolean {
        if (this === other) return true
        if (other !is Point) return false
        // Compares properties for structural equality
        return this.x == other.x && this.y == other.y
    }

}
```

Important Notes:

- When overriding equals(), you should also override hashCode()
- Functions like equals(other: Foo) don’t affect == and != operators
- Only equals(Any?) implementation affects the behavior of ==

Equality in Data Classes & Value Classes

Value classes and data classes automatically override equals(), so they implement structural equality by default. Non-data classes inherit referential equality from Any.

Data Classes

Automatically override equals()
Compare all constructor properties

Non-data Classes

Use referential equality by default
Must override equals() manually

```
// data class
data class User(val name: String)

val u1 = User("Alice")
val u2 = User("Alice")
println(u1 == u2)  // true  (structural)


// Normal Class
class Item(val name: String)

val i1 = Item("Alice")
val i2 = Item("Alice")
println(i1 == i2)  // false (referential)
```

Referential Equality

Referential equality verifies the memory addresses of two objects to determine if they are the same instance.
Checked by the === operator and its negated counterpart !==.

```
var a = "Hello"
var b = a
var c = "world"
var d = "world"
var e = a.substring(0, 1) + a.substring(1)

println(a === b)  // true
println(a === c)  // false
println(c === d)  // true
println(c === e)  // false
```

For values represented by primitive types at runtime (e.g., Int), the === check is equivalent to ==.

Floating-point Numbers Comparison

When operands are statically known to be Float or Double, equality follows the IEEE 754 Standard. For generic types (Any, Collection), the behavior differs to support total ordering.

IEEE 754 Mode

Statically typed as Float/Double:

- NaN is NOT equal to itself
- -0.0 IS equal to 0.0

Generic Mode

Not statically typed (Any, Collection):

- NaN IS equal to itself
- -0.0 is NOT equal to 0.0
- NaN is greater than POSITIVE_INFINITY

```
// IEEE 754 mode (statically typed as Double)
println(Double.NaN == Double.NaN)                // false (NaN – Not a Number – is not equal to any number)
println(0.0 == -0.0)                             // true (0.0, +0.0 and -0.0 are considered equals)

// Object equality (Double.equals)
println(Double.NaN.equals(Double.NaN))           // true
println(0.0.equals(-0.0))                        // false

// Generic mode (not statically typed as FP)
println(listOf(Double.NaN) == listOf(Double.NaN))  // true (List.equals() uses Double.equals)
println(listOf(0.0) == listOf(-0.0))             // false
```

Floating-point – Sorting Example

```
fun main() {
    val list = listOf(
        Double.NaN,
        Double.POSITIVE_INFINITY,
        0.0,
        -0.0
    )

    println(list.sorted())     // Output: [-0.0, 0.0, Infinity, NaN]
}
```

In generic/sorting context:

- -0.0 is less than 0.0
- NaN is considered greater than any other element, including POSITIVE_INFINITY

Practice Exercises

Exercise 1 – Predict the Output

✅ SOLVED

Predict the output of each println statement.
Consider the difference between structural (==) and referential (===) equality for data classes.

```
data class Person(val name: String, val age: Int)

fun main() {
    val p1 = Person("Alice", 30)
    val p2 = Person("Alice", 30)
    val p3 = p1

    println(p1 == p2)   // true  – same content
    println(p1 === p2)  // false – different instances
    println(p1 === p3)  // true  – same reference
}
```

Explanation:

Data classes auto-generate equals() comparing all constructor properties. Referential equality (===) checks if both variables point to the exact same object in memory.

Exercise 2 – Floating-point Traps

✅ SOLVED

What does each line print? Remember: IEEE 754 mode applies when operands are statically typed as Float/Double.
Generic mode applies when wrapped in collections.

```
fun main() {
    // IEEE 754 mode
    println(Double.NaN == Double.NaN)    // false
    println(0.0 == -0.0)                 // true

    // Generic mode (inside collections)
    println(listOf(Double.NaN) == listOf(Double.NaN)) // true
    println(listOf(0.0) == listOf(-0.0))        // false
}
```

Explanation:

- IEEE 754: NaN != NaN, and 0.0 == -0.0
- Generic mode (collections): NaN == NaN, and 0.0 != -0.0
- The behavior flips depending on whether the operand type is statically known

Exercise 3 – What Gets Printed?

✏️ UNSOLVED

Predict the output of each println.
Pay attention to whether each class is a data class or a regular class, and how that affects == and ===.

```
class Animal(val name: String)
data class Pet(val name: String)

fun main() {
    val a1 = Animal("Cat")
    val a2 = Animal("Cat")
    val p1 = Pet("Cat")
    val p2 = Pet("Cat")

    println(a1 == a2)   // ???
    println(p1 == p2)   // ???
    println(a1 === a2)  // ???
    println(p1 === p2)  // ???
}
```

💡 Hint: Regular classes do not override equals() – they use referential equality from Any by default.

Exercise 4 – Implement equals()

✏️ UNSOLVED

Complete the Circle class so that two circles are structurally equal when they have the same radius.
Also override hashCode() for consistency. The println at the bottom should print true.

```
class Circle(val radius: Double) {

    // TODO: override equals(other: Any?): Boolean
    // 1. Check referential equality (this === other)
    // 2. Check type with !is
    // 3. Compare radius properties

    // TODO: override hashCode(): Int
}

fun main() {
    val c1 = Circle(5.0)
    val c2 = Circle(5.0)
    println(c1 == c2)  // should print: true
}
```